



AgroMaster

Software Requirements Specification

Smart Farm Management Platform for Sri Lankan Farmers

VERSION	1.0
DATE	13 September 2026
AUTHOR	Minidu Dhananjana
STATUS	Implemented

Table of Contents

1. Introduction
 2. Overall Description
 3. System Architecture
 4. Functional Requirements
 5. Data Requirements
 6. External Interface Requirements
 7. Non-Functional Requirements
 8. Design Decisions and Rationale
 9. Testing
 10. Deployment
 11. Future Enhancements
-

1. Introduction

1.1 Purpose

This document specifies the requirements for **AgroMaster**, a web-based farm management platform for small-scale Sri Lankan farmers. It describes what the system does, how it is structured, and why particular engineering decisions were taken. It is intended for evaluators, future maintainers, and anyone extending the system.

1.2 Scope

AgroMaster lets a farmer record every crop from planting to harvest, log the costs incurred against it, record the revenue it produced, and see whether that crop actually made money. On top of that record it layers three assistive features: an AI advisor that answers farming questions in context, live weather for the farm's location, and email reminders for time-sensitive tasks.

The system is explicitly **single-farmer per account**. There is no organisation hierarchy, no shared farms, and no role system. Every record belongs to exactly one user, and ownership is enforced on every query.

Out of scope for version 1.0: marketplace or trading features, direct buyer connections, offline mobile operation, SMS notifications, and multi-language support.

1.3 Definitions

TERM	MEANING
Crop	One planting of one variety in one field, tracked from sowing to harvest
Expense	A cost recorded against a crop, in one of eight categories
Harvest	A recorded yield: quantity, unit, price per unit, and buyer
Reminder	A dated task attached to a crop, optionally emailed in advance
Digest	The single daily email summarising a farmer's upcoming reminders
P&L	Profit and loss: harvest revenue minus recorded expenses
JWT	JSON Web Token, the stateless session credential
Maha / Yala	The two Sri Lankan cultivation seasons

1.4 References

- Repository: github.com/minidu10/cultivation-help-app
- API documentation: Swagger UI at [/swagger-ui.html](#)
- Setup and operations: [README.md](#)

2. Overall Description

2.1 Product Perspective

AgroMaster is a self-contained three-tier web application. It depends on four external services, none of which are on the critical path for core record-keeping:

SERVICE	USED FOR	IF UNAVAILABLE
PostgreSQL	All persistent data	System is down
SMTP relay	Verification codes, reminders	Registration and reset blocked; existing users unaffected
OpenAI-compatible LLM	AI advisor	Advisor returns 503; rest works
OpenWeather	Weather widgets	Weather panels fail; rest works
Google Identity	Optional sign-in	Button hidden; password login works

This degradation is deliberate. A farmer must be able to record an expense even when an API key has expired.

2.2 User Characteristics

The target user is a small-scale farmer in Sri Lanka with a smartphone or shared computer, basic literacy, and limited tolerance for complexity. Design consequences:

- Every monetary value is displayed in Sri Lankan Rupees
- The interface is fully responsive; phone width is the assumed default
- Emails are plain text, short, and written in simple language
- Reminders arrive **before** a task is due, not when it is already overdue

2.3 Constraints

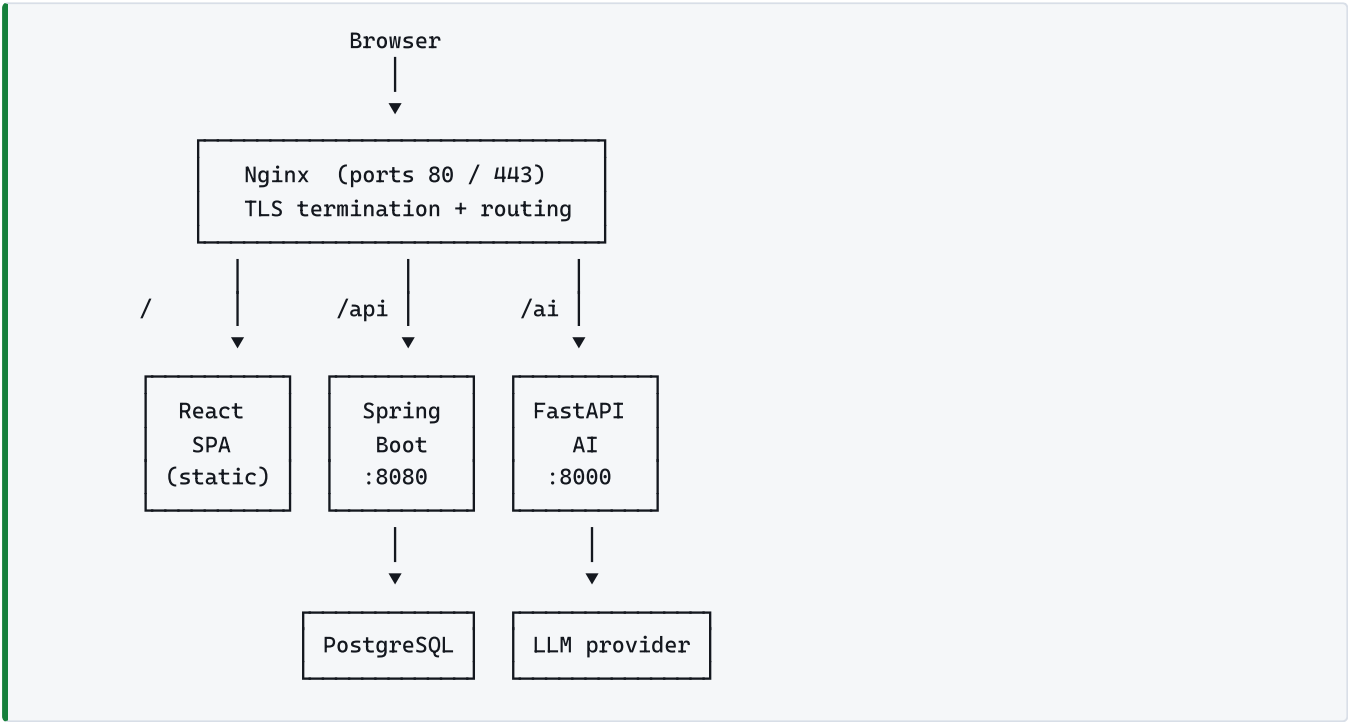
CONSTRAINT	CONSEQUENCE
Near-zero hosting budget	Free-tier services throughout; Docker Compose on a single host
No dedicated ops staff	Schema migrations run automatically at boot
Unreliable connectivity	Stateless JWT auth; no server-side session state
Sri Lanka timezone (UTC+5:30)	Container clock pinned to <code>Asia/Colombo</code>

2.4 Assumptions and Dependencies

- Each farmer owns a working email address, verified at registration
 - A single application instance is deployed (the reminder scheduler assumes no concurrent runner)
 - The database is managed externally in production and is backed up by the provider
-

3. System Architecture

3.1 Component Overview



3.2 Technology Stack

LAYER	TECHNOLOGY	VERSION
Frontend	React, Vite, React Router, Recharts	React 19, Vite 8
Backend	Spring Boot, Spring Security, Flyway, JWT	Spring Boot 4, Java 17
AI service	FastAPI, OpenAI SDK	Python 3.12
Database	PostgreSQL	16
Web server	Nginx	alpine
Orchestration	Docker Compose	—

3.3 Codebase Size

COMPONENT	LINES
Backend (Java)	~3,850
Frontend (JSX/JS)	~4,215
AI service (Python)	~190

3.4 Environment Strategy

A **single** `docker-compose.yml` serves both local development and production. `COMPOSE_PROFILES` in `.env` selects which services start, so the deployment command is identical in both environments.

SERVICE	PROFILE LOCAL	PROFILE PROD
db (PostgreSQL)	starts	not started — managed database
mailpit (mail catcher)	starts	not started — real SMTP relay
backend	starts	starts
ai-service	starts	starts
frontend (Nginx + TLS)	not started — Vite dev server	starts

4. Functional Requirements

4.1 Account Management

ID	REQUIREMENT	PRIORITY
FR-1.1	The system shall require email verification before creating an account	High
FR-1.2	The system shall email a 6-digit code valid for 10 minutes	High
FR-1.3	The system shall invalidate a code after 5 incorrect attempts	High
FR-1.4	The system shall enforce a 60-second cooldown between code requests	High
FR-1.5	The system shall reject registration without a confirmed, unspent verification	High
FR-1.6	The system shall enforce password rules server-side: minimum 8 characters, upper and lower case, a digit, and a symbol	High
FR-1.7	The system shall send a confirmation email on successful registration	Medium
FR-1.8	The system shall permit password reset using the same code mechanism	High
FR-1.9	The system shall return an identical response to password reset requests whether or not the account exists	High
FR-1.10	The system shall support account creation and sign-in through Google	Medium
FR-1.11	The system shall link a Google identity to an existing account only when Google reports the email as verified	High

Registration flow

```

Enter email → POST /auth/send-code → 6-digit code emailed
Enter code → POST /auth/verify-code → verification confirmed
Enter details → POST /auth/register → account created, JWT issued

```

Verification precedes account creation, so no unverified rows are ever persisted and a failed email delivery cannot lock a user out of an account they already own.

4.2 Crop Management

ID	REQUIREMENT	PRIORITY
FR-2.1	The system shall allow creating a crop with name, variety, field location, size in acres, planting date and expected harvest date	High
FR-2.2	The system shall track crop status as PLANTED, GROWING, READY_TO_HARVEST, HARVESTED or FAILED	High
FR-2.3	The system shall allow editing and deleting a crop	High
FR-2.4	The system shall cascade deletion to the crop's expenses, harvests and reminders	High
FR-2.5	The system shall return only crops belonging to the authenticated user	High

4.3 Expense Tracking

ID	REQUIREMENT	PRIORITY
FR-3.1	The system shall record an expense against a crop with category, description, amount and date	High
FR-3.2	The system shall support categories: SEEDS, FERTILIZER, PESTICIDES, LABOR, IRRIGATION, EQUIPMENT, TRANSPORT, MISCELLANEOUS	High
FR-3.3	The system shall total expenses per crop	High

4.4 Harvest Records

ID	REQUIREMENT	PRIORITY
FR-4.1	The system shall record a harvest with date, quantity, unit, price per unit and buyer	High
FR-4.2	The system shall support units: KG, TONNE, BUSHEL, POUND, LITER	High
FR-4.3	The system shall compute revenue as quantity × price per unit	High
FR-4.4	The system shall support multiple harvests per crop	High

4.5 Profit Analysis

ID	REQUIREMENT	PRIORITY
FR-5.1	The system shall compute per-crop profit as total revenue minus total expenses	High
FR-5.2	The system shall classify each crop as PROFIT or LOSS	High
FR-5.3	The system shall present profit and loss visually on the dashboard	Medium

4.6 Reminders and Notifications

ID	REQUIREMENT	PRIORITY
FR-6.1	The system shall allow reminders against a crop with title, type, date-time and note	High
FR-6.2	The system shall support types: FERTILIZER, IRRIGATION, PEST_CONTROL, HARVEST, OTHER	Medium
FR-6.3	The system shall email advance notice 2 days and 1 day before a reminder falls due	High
FR-6.4	The system shall send at most one digest email per farmer per day, covering all their upcoming tasks	High
FR-6.5	The system shall never send the same advance notice twice	High
FR-6.6	The system shall exclude completed and disabled reminders from notification	High
FR-6.7	The system shall allow a farmer to disable reminder emails from Settings	High

Rationale for FR-6.3 and FR-6.4. An alert at the moment a task falls due is too late to act on; fertiliser must be bought the day before. And one email per reminder floods the inbox during a busy week, which trains farmers to ignore the messages entirely. Grouping into a single daily digest keeps the channel credible.

4.7 AI Advisory

ID	REQUIREMENT	PRIORITY
FR-7.1	The system shall answer free-text farming questions, informed by the farmer's crop context	Medium
FR-7.2	The system shall generate performance insights for a crop from its financial record	Medium
FR-7.3	The system shall express costs in Sri Lankan Rupees	Medium
FR-7.4	The system shall support any OpenAI-compatible provider through configuration alone	Medium

4.8 Weather

ID	REQUIREMENT	PRIORITY
FR-8.1	The system shall show current conditions for the farmer's location	Medium
FR-8.2	The system shall show a multi-day forecast	Medium
FR-8.3	The system shall proxy weather requests server-side so the API key is never exposed to the browser	High

4.9 Settings

ID	REQUIREMENT	PRIORITY
FR-9.1	The system shall allow editing name, phone and city	Medium
FR-9.2	The system shall not allow changing the account email	Medium
FR-9.3	The system shall support Light, Dark and System themes	Low
FR-9.4	The system shall expose the reminder email opt-out	High

5. Data Requirements

5.1 Entity Relationships

```
users (1) —< (N) crops (1) —< (N) expenses
                        |
                        |—< (N) harvests
                        |
                        |—< (N) reminders (1) —< (N) reminder_notifications
email_verifications      (standalone – no user yet at registration)
```

All child relationships cascade on delete.

5.2 Tables

TABLE	PURPOSE
users	Accounts, preferences, auth provider
crops	Plantings owned by a user
expenses	Costs against a crop
harvests	Yields and revenue against a crop
reminders	Dated tasks against a crop
reminder_notifications	Which advance notice has been emailed
email_verifications	Registration and reset codes
flyway_schema_history	Migration ledger

5.3 Schema Evolution

Schema changes are applied exclusively through Flyway migrations. Hibernate's `ddl-auto` is set to `none`, so the schema is never altered implicitly.

MIGRATION	CHANGE
V0	Core schema: users, crops, expenses, harvests
V1	Reminders and AI insight flag
V2	User city
V3	Theme and desktop-mode preferences
V4	Performance indexes on foreign keys
V5	Email verification codes
V6	Reminder advance notifications
V7	Reminder email opt-out
V8	Google sign-in columns

5.4 Indexing

PostgreSQL does not index foreign keys automatically, and every repository query filters on one. V4 adds composite indexes ordered to satisfy both the filter and the sort each query requests:

INDEX	SERVES
<code>idx_crops_user_created</code>	Crop list for a user, newest first
<code>idx_expenses_crop_date</code>	Expenses per crop, and the expense total
<code>idx_harvests_crop_date</code>	Harvests per crop, and the revenue total
<code>idx_reminders_crop_at</code>	Reminders per crop
<code>idx_reminders_due</code>	Partial index over pending reminders only

5.5 Data Integrity

RULE	ENFORCED BY
Unique email per account	<code>UNIQUE</code> constraint on <code>users.email</code>
Unique Google identity	Partial unique index on <code>users.google_id</code>
No duplicate reminder notice	<code>UNIQUE (reminder_id, lead_days)</code>
Orphan prevention	<code>ON DELETE CASCADE</code> on every child table

Duplicate-notification prevention is a **database constraint rather than application logic**, so a scheduler restart or a second application instance cannot produce a duplicate email.

6. External Interface Requirements

6.1 REST API

All endpoints are prefixed `/api`. Every endpoint except those listed as public requires a `Bearer` JWT.

Authentication — public

METHOD	PATH	PURPOSE
GET	/auth/config	Public sign-in options
POST	/auth/send-code	Email a registration code
POST	/auth/verify-code	Confirm the code
POST	/auth/register	Create the account
POST	/auth/login	Obtain a JWT
POST	/auth/google	Sign in with a Google ID token
POST	/auth/forgot-password	Email a reset code
POST	/auth/reset-password	Set a new password

Crops, expenses, harvests, reminders — authenticated

METHOD	PATH
GET, POST	/crops
GET, PUT, DELETE	/crops/{id}
GET, POST	/crops/{cropId}/expenses
DELETE	/crops/{cropId}/expenses/{expenseId}
GET	/crops/{cropId}/expenses/profit-loss
GET, POST	/crops/{cropId}/harvests
DELETE	/crops/{cropId}/harvests/{harvestId}
GET, POST	/crops/{cropId}/reminders
PUT, DELETE	/crops/{cropId}/reminders/{reminderId}
GET	/reminders/due

User and weather — authenticated

METHOD	PATH
GET, PUT	/users/me
GET	/weather/current , /weather/current/by-location
GET	/weather/forecast , /weather/forecast/by-location

AI service — served at /ai

METHOD	PATH
POST	/ai/ask
POST	/ai/insights
GET	/ai/health

6.2 Error Format

Every error returns a consistent JSON body:

```
{
  "timestamp": "2026-09-13T10:24:11.482Z",
  "status": 400,
  "error": "Bad Request",
  "message": "Incorrect code. 3 attempt(s) remaining.",
  "path": "/api/auth/verify-code"
}
```

6.3 Configuration

All configuration is supplied through environment variables in a single root `.env`. The frontend requires no environment file: it calls relative paths that Nginx (production) and the Vite dev server (development) proxy identically.

VARIABLE	PURPOSE
COMPOSE_PROFILES	local or prod
DB_URL , DB_USERNAME , DB_PASSWORD	Database connection
JWT_SECRET	Token signing key, 32+ characters
MAIL_*	SMTP transport
AI_API_KEY , AI_BASE_URL , AI_MODEL	LLM provider
GOOGLE_CLIENT_ID	Google sign-in; blank disables the feature
OPENWEATHER_API_KEY	Weather data
TZ	Container timezone
REMINDER_*	Digest schedule and lead days

7. Non-Functional Requirements

7.1 Security

ID	REQUIREMENT	IMPLEMENTATION
NFR-1.1	Passwords shall never be stored in recoverable form	BCrypt
NFR-1.2	Verification codes shall not be stored in plaintext	SHA-256 of <code>email:code</code>
NFR-1.3	Sessions shall be stateless	JWT, 24-hour expiry
NFR-1.4	Codes shall resist brute force	10-minute expiry, 5 attempts, single use
NFR-1.5	Authentication shall not reveal which emails are registered	Identical response for unknown email on login and password reset
NFR-1.6	Third-party tokens shall be cryptographically verified	Google ID token signature, issuer, audience and expiry checked against Google's published keys
NFR-1.7	API keys shall never reach the browser	Weather and AI proxied server-side
NFR-1.8	Secrets shall never enter version control	<code>.env</code> gitignored; only <code>.env.example</code> tracked
NFR-1.9	Ownership shall be enforced at the query level	Every repository method filters by user id
NFR-1.10	Transport shall be encrypted in production	TLS via Let's Encrypt

Note on NFR-1.5. An endpoint that answers differently for a registered and an unregistered email is an account-enumeration oracle. Both the login and forgot-password paths return one indistinguishable response.

7.2 Performance

ID	REQUIREMENT
NFR-2.1	All foreign-key columns used in queries shall be indexed
NFR-2.2	The reminder digest shall fetch crop and owner eagerly, avoiding N+1 queries
NFR-2.3	Outbound email shall not block an HTTP response where the user is not waiting on it
NFR-2.4	The database connection pool shall tolerate an unavailable database at startup

7.3 Reliability

ID	REQUIREMENT
NFR-3.1	Failure of an external service shall not prevent core record-keeping
NFR-3.2	A single failed email shall not abort a batch of notifications
NFR-3.3	A notification shall be recorded as sent only after the send succeeds
NFR-3.4	Schema migrations shall be applied automatically and idempotently
NFR-3.5	Local data shall survive container restarts

7.4 Usability

ID	REQUIREMENT
NFR-4.1	All screens shall be usable at 400 px width
NFR-4.2	Validation feedback shall be immediate and specific
NFR-4.3	Rate-limit cooldowns shall be shown as a live countdown, not an unexplained error
NFR-4.4	Monetary values shall be displayed in Sri Lankan Rupees

7.5 Maintainability

ID	REQUIREMENT
NFR-5.1	One compose file shall serve all environments
NFR-5.2	Provider choice shall be configuration, not code
NFR-5.3	Schema changes shall go through versioned migrations only
NFR-5.4	The frontend shall build with zero lint errors

8. Design Decisions and Rationale

8.1 Verification before account creation

Decision. The email is proven before any user row exists, rather than creating an unverified account and gating login.

Rationale. Gating login on a verification flag means a mail delivery failure locks a user out of an account they already own, requiring a resend endpoint and support path. Verifying first means the failure mode is simply "cannot register yet" — recoverable by retrying, with no orphaned rows and no partial state.

8.2 Codes rather than magic links

Decision. A 6-digit code the user types, not a clickable link.

Rationale. Codes work when mail is read on a different device from the one registering, which is common where a phone is shared. The trade-off is that a 6-digit code has only one million possibilities and is brute-forceable, so it is bound by a 10-minute expiry, a 5-attempt limit and single use. Without the attempt limit the mechanism would be decorative.

8.3 Daily digest rather than per-reminder email

Decision. One message per farmer per day, covering everything upcoming.

Rationale. A farmer with five tasks tomorrow receiving five separate emails will stop reading them. Running the job once daily makes more than one message per farmer per day structurally impossible, rather than something the code must remember not to do.

8.4 A table for sent notifications

Decision. `reminder_notifications` with a unique constraint, rather than a `notified_at` timestamp on the reminder.

Rationale. Each reminder is announced more than once (two days out, then one day out), which a single timestamp cannot express. Placing the uniqueness rule in the database rather than in job logic means a restart, an overlapping run, or a second application instance cannot produce a duplicate email.

8.5 One OpenAI-compatible client

Decision. A single client with configurable base URL and model, rather than one integration per provider.

Rationale. Gemini, Groq, OpenAI, OpenRouter and Ollama all expose the same protocol, so per-provider SDKs add dependency surface without adding capability. This was not theoretical: three concurrent SDKs produced a version conflict that crash-looped the service at import time. Provider choice is now two environment variables, and a retired model is a config change rather than a code change.

8.6 Profiles rather than two compose files

Decision. One `docker-compose.yml` ; `COMPOSE_PROFILES` selects the service set.

Rationale. Parallel compose files drift. With profiles the deployment command is identical in both environments, and the difference between them is visible in one `.env`.

8.7 Client ID served by the API

Decision. The Google client ID is returned by `GET /auth/config` rather than compiled into the frontend bundle.

Rationale. It keeps the frontend free of any environment file, and changing the ID requires no rebuild. It also allows the sign-in button to hide itself cleanly when Google is not configured.

8.8 Timezone pinned explicitly

Decision. The backend container runs on `Asia/Colombo`.

Rationale. Reminder times are stored as zoneless local times. A container defaulting to UTC would fire a 07:00 reminder at 01:30 local time — a silent five-and-a-half-hour error in the system's most time-sensitive feature.

9. Testing

9.1 Verified Behaviour

AREA	VERIFIED
Registration	Code delivery, verification, account creation, welcome email
Brute force	Code dies after 5 wrong attempts; counter persists across the rejection
Rate limiting	Resend blocked within the 60-second cooldown
Password policy	Weak passwords rejected server-side, not only in the browser
Enumeration	Unknown email returns the same response as a known one
Password reset	Code, reset, login with the new password; old password rejected
Google sign-in	Forged <code>alg:none</code> and malformed tokens both rejected with 401
Null password	Google-only account cannot sign in with a password
Reminder digest	Correct crops selected by lead time; completed and out-of-window excluded
Duplicate suppression	Repeated scheduler runs produce no second email
Opt-out	No email sent when the farmer has disabled reminders
Indexes	<code>EXPLAIN ANALYZE</code> confirms index scans, not sequential scans
Migrations	V0–V8 apply cleanly to an empty database

9.2 Known Gaps

Automated test coverage is the principal weakness of version 1.0. The backend contains only a context-load test; there are no unit tests, no integration tests, and no frontend tests. All verification above was performed manually against a running system.

Closing this gap is the highest-priority engineering task, in this order:

1. Service-layer unit tests for `VerificationService` (the brute-force and single-use guarantees)
2. Integration tests for the authentication endpoints
3. A scheduler test asserting no duplicate notification
4. Component tests for the multi-step registration form

10. Deployment

10.1 Local

```
cp .env.example .env
docker compose up -d --build
cd frontend-web && npm install && npm run dev
```

SERVICE	ADDRESS
Frontend	http://localhost:5173
Backend API	http://localhost:8080
Swagger UI	http://localhost:8080/swagger-ui.html
AI service	http://localhost:8000
Mail inbox (Mailpit)	http://localhost:8025
PostgreSQL	localhost:5434

No mail leaves the machine locally: Mailpit accepts every message and displays it in a web inbox.

10.2 Production

A single host runs all three containers behind Nginx, with TLS certificates from Let's Encrypt and a managed PostgreSQL instance. Deployment is automated: a push to `main` triggers a GitHub Actions workflow that connects over SSH, pulls, and rebuilds.

Keeping the database off the application host means compute can be migrated between providers without moving data.

11. Future Enhancements

ENHANCEMENT	VALUE
Automated test suite	Closes the largest quality gap
Crop disease diagnosis from photographs	The configured AI provider is already multimodal; likely the highest-value feature available
Sinhala and Tamil interfaces	Substantially widens the addressable user base
Season-over-season comparison	Turns single-season records into trends
Export to PDF or spreadsheet	Farmers need records for loan and subsidy applications
Route-level code splitting	Reduces the initial bundle, material on rural connections

End of document.